

Les WebSockets

1. Introduction : Qu'est-ce qu'un WebSocket ?

Le **WebSocket** est une technologie permettant une communication en temps réel entre un navigateur et un serveur.

Pour comprendre simplement son fonctionnement, on peut comparer :

- le **HTTP classique** à un échange de lettres,
 - le **WebSocket** à un appel téléphonique permanent.
-

Fonctionnement classique : HTTP

Avec le protocole HTTP :

1. Le navigateur envoie une requête.
2. Le serveur répond.
3. La connexion se ferme.

Si l'application veut savoir si quelque chose a changé :

- elle doit refaire une nouvelle requête,
- encore et encore.

C'est ce qu'on appelle le **polling**.

Fonctionnement WebSocket

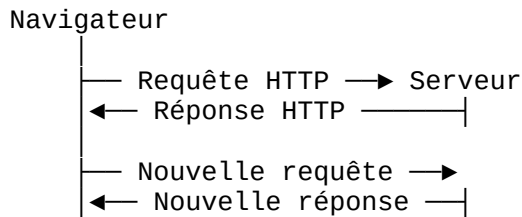
Avec un WebSocket :

1. Le navigateur ouvre une connexion unique avec le serveur.
2. La connexion reste ouverte en continu.
3. Le serveur et le client peuvent communiquer instantanément à tout moment.

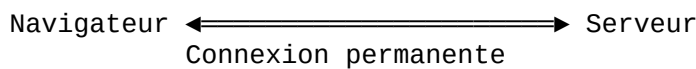
Le serveur peut donc envoyer des données sans attendre qu'on lui demande.

Schéma de comparaison

HTTP classique



WebSocket



2. Les avantages des WebSockets

Temps réel instantané

Les données apparaissent immédiatement à l'écran dès qu'un événement se produit.

Exemples :

- nouveau message,
- notification,
- déplacement d'un joueur,
- variation d'un prix en bourse.

Aucun rechargement n'est nécessaire.

Réduction des requêtes inutiles

Avec HTTP classique, le navigateur demande constamment :

“Y a-t-il du nouveau ?”

Avec WebSocket :

- le serveur parle uniquement quand il a réellement quelque chose à envoyer.

Cela réduit :

- le trafic réseau,

- les requêtes inutiles,
 - la charge serveur à long terme.
-

Communication bidirectionnelle

Le client et le serveur peuvent envoyer des informations simultanément.

Cela permet :

- des applications interactives,
 - des jeux en ligne,
 - des systèmes collaboratifs.
-

3. Les inconvénients

Consommation mémoire côté serveur

Chaque utilisateur garde une connexion ouverte.

Si :

- 10 000 utilisateurs sont connectés,
- alors le serveur doit maintenir 10 000 connexions actives.

Cela demande :

- plus de RAM,
 - une bonne infrastructure,
 - des serveurs bien optimisés.
-

Gestion des déconnexions

Si la connexion internet coupe :

- le WebSocket est interrompu.

L'application doit alors :

- détecter la coupure,
 - tenter une reconnexion automatique.
-

Complexité inutile pour certains projets

Le WebSocket n'est pas utile partout.

Pour :

- un blog,
- un site vitrine,
- une page statique,

le HTTP classique est largement suffisant.

4. Pourquoi utiliser les WebSockets ?

Les utilisateurs modernes attendent des applications instantanées.

Aujourd'hui :

- personne ne veut actualiser la page manuellement,
- les applications doivent réagir automatiquement.

Les WebSockets permettent de créer des interfaces :

- vivantes,
 - réactives,
 - modernes.
-

5. Quand utiliser les WebSockets ?

Les WebSockets sont utiles dès qu'une information doit être transmise en temps réel.

Messageries instantanées

Exemples :

- Discord
- Slack
- WhatsApp

Les messages apparaissent immédiatement.

Notifications en direct

Exemples :

- likes,
 - messages privés,
 - alertes.
-

Jeux multijoueurs

Dans les jeux en ligne :

- les déplacements,
- les tirs,
- les actions des joueurs

doivent être synchronisés instantanément.

Tableaux financiers

Les plateformes de :

- bourse,
- cryptomonnaies,
- trading

mettent à jour les valeurs en temps réel.

Travail collaboratif

Exemple :

- Google Docs

Les utilisateurs voient les modifications des autres en direct.

6. Fonctionnement technique dans Vue.js

Dans une application Vue.js, l'utilisation d'un WebSocket suit généralement 4 étapes.

Les développeurs utilisent souvent la bibliothèque :

[Socket.IO](https://socket.io/)

Étape 1 — Initialisation de la connexion

Quand le composant Vue.js apparaît :

- on ouvre la connexion avec le serveur.

Exemple

```
import { io } from "socket.io-client";  
  
const socket = io("http://localhost:3000");
```

Étape 2 — Écoute des événements

Le navigateur attend les messages envoyés par le serveur.

Exemple

```
socket.on("new-message", (data) => {  
  console.log(data);  
});
```

Étape 3 — Envoi de données

Le client peut envoyer des informations au serveur.

Exemple

```
socket.emit("send-message", {  
  message: "Bonjour"  
});
```

Étape 4 — Fermeture de la connexion

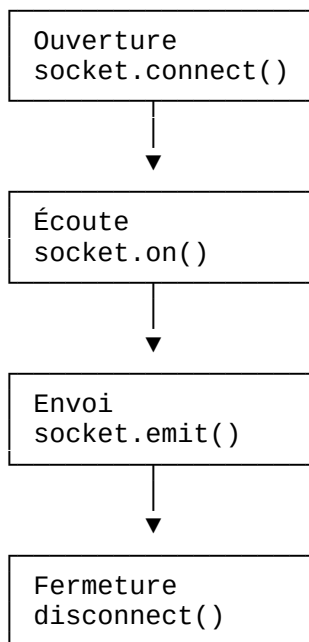
Quand l'utilisateur quitte la page :

- il faut fermer la connexion.

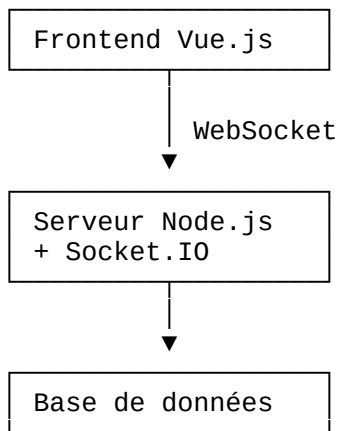
Exemple

```
socket.disconnect();
```

Cycle complet d'un WebSocket



7. Architecture simplifiée d'un système WebSocket



8. Les bonnes pratiques

À faire

- ✓ Fermer les connexions inutilisées
- ✓ Mettre en place une reconnexion automatique
- ✓ Sécuriser les échanges avec HTTPS/WSS
- ✓ Vérifier les permissions côté serveur
- ✓ Limiter le nombre d'événements envoyés

À éviter

- ✗ Garder des connexions fantômes ouvertes
 - ✗ Envoyer trop de données inutilement
 - ✗ Utiliser WebSocket pour un simple site vitrine
 - ✗ Faire confiance uniquement au frontend
-

9. Différence entre HTTP et WebSocket

HTTP	WebSocket
Requête → Réponse	Connexion permanente
Communication ponctuelle	Temps réel
Simple à mettre en place	Plus complexe
Idéal pour sites classiques	Idéal pour applications interactives
Plus de requêtes réseau	Moins de requêtes répétitives

10. Conclusion

Les WebSockets sont aujourd'hui essentiels pour les applications modernes nécessitant du temps réel.

Ils permettent :

- une communication instantanée,
- des applications dynamiques,
- une expérience utilisateur fluide,
- des échanges bidirectionnels performants.

Cependant, ils demandent :

- une infrastructure adaptée,
- une bonne gestion mémoire,
- une sécurisation rigoureuse.

Dans les projets modernes utilisant Vue.js, les WebSockets sont devenus indispensables pour :

- les chats,
 - les jeux multijoueurs,
 - les notifications,
 - et toutes les applications interactives en temps réel.
-

Sources utiles

Documentation Socket.IO

[Documentation officielle Socket.IO](#)

Documentation MDN WebSocket

[MDN WebSockets API](#)